

```

public class Main {
    public static void main(String[] args) {
        Room room = new Room(new char[][] {
            {'#', '#', '#', '#', '#', '#', '#', '#', '#',
            '#', '#', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', '#', '#', '#', '#', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', '#', ' ', ' ', ' ', '#', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', '#', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', '#', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', ' ', ' ', ' ', ' ', ' ', '#', ' ', ' ', ' ', ' ', ' ',
            ' ', ' ', '#'},
            {'#', '#', '#', '#', '#', '#', '#', '#', '#', '#',
            '#', '#', '#'}
        });

        System.out.println("Initial room layout:");
        room.print();
        Position start = new Position(6, 7);
        Position target = new Position(1, 1);

        System.out.println("\nDistance matrix:");
        room.printDistanceMatrix(target);
        System.out.println("\nShortest path:");
        room.printShortestPath(start, target);
    }
}

```

```

import java.util.LinkedList;

public class Room {
    private char[][] layout;

    public Room(char[][] layout) {
        this.layout = layout;
    }

    public boolean isAccessible(Position position) {
        int x = position.getX();
        int y = position.getY();

        if (x < 0 || x >= layout.length) {
            return false;
        }
        if (y < 0 || y >= layout[0].length) {
            return false;
        }
        if (layout[x][y] == '#') {
            return false;
        }
        return true;
    }

    public void print() {
        for (int x = 0; x < layout.length; x++) {
            for (int y = 0; y < layout[x].length; y
++) {
                System.out.print(layout[x][y]);
            }
            System.out.println();
        }
    }

    public int[][] getDistanceMatrix(Position target) {
        int[][] distance = new int[layout.length][
layout[0].length];

        for (int x = 0; x < distance.length; x++) {
            for (int y = 0; y < distance[x].length; y

```

```

++) {
    distance[x][y] = -1;
}
}
distance[target.getX()][target.getY()] = 0;
LinkedList<Position> list = new LinkedList<>();
list.add(target);

while (list.isEmpty() == false) {
    Position current = list.remove();
    Position[] neighbors = current.getNeighbors
());

    for (int i = 0; i < neighbors.length; i
++) {
        Position neighbor = neighbors[i];
        if (isAccessible(neighbor)) {
            int x = neighbor.getX();
            int y = neighbor.getY();

            if (distance[x][y] == -1) {
                distance[x][y] = distance[
current.getX()][current.getY()] + 1;
                list.add(neighbor);
            }
        }
    }
}
return distance;
}

public void printDistanceMatrix(Position target) {
    int[][] distance = getDistanceMatrix(target);
    for (int x = 0; x < distance.length; x++) {
        for (int y = 0; y < distance[x].length; y
++) {
            if (layout[x][y] == '#') {
                System.out.print("## ");
            } else {
                System.out.printf("%2d ", distance[
x][y]);

```

```

        }
    }
    System.out.println();
}

public void printShortestPath(Position start,
Position target) {
    int[][] distance = getDistanceMatrix(target);
    Position current = start;
    System.out.print("(" + current.getX() + "|" +
current.getY() + ")");

    while (current.getX() != target.getX() ||
current.getY() != target.getY()) {
        Position next = null;
        Position[] neighbors = current.getNeighbors
();

        for (int i = 0; i < neighbors.length; i
++) {
            Position neighbor = neighbors[i];
            if (isAccessible(neighbor)) {
                int x = neighbor.getX();
                int y = neighbor.getY();
                int currentDistance = distance[
current.getX()][current.getY()];

                if (distance[x][y] ==
currentDistance - 1) {
                    next = neighbor;
                }
            }
        }
        if (next == null) {
            System.out.println(" Kein Weg gefunden
.");

            return;
        }
        current = next;
        System.out.print(" -> (" + current.getX

```

```
() + "|" + current.getY() + ")");  
    }  
    System.out.println();  
}  
}
```

```
import java.util.Objects;

public class Position {
    private int x;
    private int y;

    public Position(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public int getX() {
        return x;
    }
    public int getY() {
        return y;
    }
    public Position[] getNeighbors() {
        return new Position[] {
            new Position(x - 1, y),
            new Position(x + 1, y),
            new Position(x, y - 1),
            new Position(x, y + 1)
        };
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Position position)) return
false;
        return x == position.x && y == position.y;
    }

    @Override
    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```